

SLAM en 3D avec rtabmap

Documentation Technique

April 2026

Table des matières

1	Introduction	2
1.1	Présentation	2
1.2	Prérequis	2
2	Navigation Autonome avec Nav2	2
2.1	Packages nécessairent	2
2.2	Conversion PointCloud vers LaserScan	3
2.3	Cartographie (SLAM)	4
2.4	Lancement de la navigation	4
2.5	Enregistrement de la carte	5
3	Utilisation de RTABMAP	7
3.1	Intallation des packages	7
3.2	Paramétrage RTABMAP	7
3.3	Première utilisation RTABMAP	10
4	Navigation	10
4.1	Identification de problèmes	10
4.2	Fonctionnement :	11
4.3	Avec IMU :	12

1 Introduction

1.1 Présentation

Dans cette section nous allons utiliser le robot et le TP de prise en main pour afficher un environnement en 3D. A l'aide de rtabmap qui aura besoin du Lidar 3D (le RS-Helios) et de la caméra RGB-D (la Realsense D435).

1.2 Prérequis

Au démarrage, le robot initialise la caméra avec leurs paramètres et leurs paramètres. Il va donc falloir modifier ses paramètres pour qu'on puisse avoir des images RGB et de de profondeur qui match.

Dans /roscat_ws/src/gr_bringup/launch/component_realsense.launch.py ajouter :

```
launch_arguments={
    'initial_reset': "True",
    'depth_module.depth_profile': "640,480,30",
    # recadrage de la camera de profondeur
    'rgb_camera.profile': "640,480,30",
    # valeur naturel mais pour etre sur
    'align_depth.enable': "true",
    # alignement du champ de vision de la camera
    # de profondeur base sur la camera rgb
}.items()
```

Le dernier paramètre va créer un nouveau topic qui publie les valeur de la caméra de profondeur alignées a celle de la caméra RGB. Vous pouvez vérifier. Dans rviz, faite ADD -> By Topic -> camera et choisissez la camera rgb, la camera de profondeur alignée et la caméra de profondeur non alignée. Si le topic de la caméra aligné d'apparait pas, il vous faut relancer le main.launch.py. Si elle n'apparait pas ou que le launch file ne fonctionne pas, vous pouvez redémarrer le robot.

2 Navigation Autonome avec Nav2

Cette partie s'effectue exclusivement dans les dockers. N'oubliez pas de sourcer votre environnement si ce n'est pas déjà fait : source /opt/ros/humble/setup.bash.

2.1 Packages nécessaires

```
# Installation des outils
sudo apt update
sudo apt install -y \
    ros-humble-navigation2 \
    ros-humble-nav2-bringup \
    ros-humble-pointcloud-to-laserscan \
    ros-humble-slam-toolbox \
    python3-colcon-common-extensions \
    python3-rosdep
```

```

cd ~/roskit_ws/src

# Clonage de Navigation2 (Branche Humble)
git clone https://github.com/ros-navigation/navigation2.git -b humble

cd ~/roskit_ws
sudo rosdep init
rosdep update
rosdep install --from-paths src --ignore-src -r -y

# On desactive les flags qui pourraient forcer le multi-coeur
unset MAKEFLAGS

# Compilation globale limitee a 1 worker, les packets etant tres massif, il
# pourrait faire crash le robot.
colcon build --symlink-install \
  --cmake-args -DCMAKE_BUILD_TYPE=Release \
  --parallel-workers 1

# On lance tous les programmes
source install/setup.bash
ros2 launch gr_bringup main.launch.py

```

Vérifiez que le robot publie. Dans noVNC, regarder si la camera renvoie bien des images et le lidar aussi. Si tout est static vous allez certainement devoir relancer le robot. Vous n'avez pas a tout retélécharger. Sinon garder le terminal avec le dernier launch.

Vous pouvez vérifier si les packets sont bien installés dans la partie docker (voir figure 7.c, faite défiler, puis `home/user/roskit_ws/src` vous trouverez vos packages).

2.2 Conversion PointCloud vers LaserScan

Le Lidar du robot publie un nuage de points 3D. Nav2 nécessite du 2D. On veut donc transformer les points 3D. En effet, il publie dans `/rslidar_points` un nuage de points (visible dans rviz). Le problème est qu'il publie des points au sol. Ces points pourront être considéré comme des `murs au moment de la construction de la carte`. Ainsi, pour obtenir notre carte 2D nous ne considérerons que les points compris entre -0.1m et 0.5m par rapport au Lidar. Ces points sont projetés sur un plan 2D qui va permettre de créer la carte.

```

source /opt/ros/humble/setup.bash
ros2 run pointcloud_to_laserscan pointcloud_to_laserscan_node \
  --ros-args \
  -r cloud_in:=/rslidar_points \
  -p target_frame:=base_link \
  -p min_height:=-0.1 \
  -p max_height:=0.5 \
  -p range_min:=0.1 \
  -p range_max:=30.0

```

Vérifiez la publication des données sur `/scan` avec `ros2 topic echo /scan`.

2.3 Cartographie (SLAM)

Afin de naviguer, nav2 à besoin d'une carte. Pour l'instant nous n'en avons pas. Nous allons utiliser slam_toolbox pour en faire une à partir de l'odométrie du robot et du laserscan récupéré juste au-dessus. L'odométrie permet d'estimer la position du robot très rapidement mais reste imprécise, la node la corrige avec les valeurs du laserscan. On utilise le mode **online asynchrone** afin de créer notre carte de façon dynamique. Le robot peut bouger tout en créant la carte. Si le processeur est très sollicité, le programme préférera ignorer quelques points laser plutôt que de retarder l'estimation de la position.

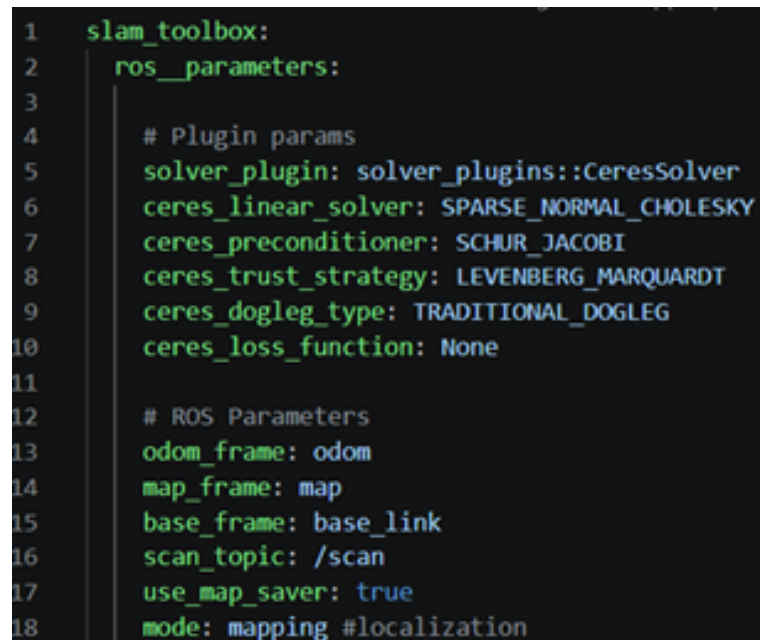
Utilisez slam_toolbox pour générer la carte :

```
ros2 launch slam_toolbox online_async_launch.py
```

Il se peut que vous rencontriez des problèmes liés à l'odométrie. Veuillez vérifier que la base_frame est bien base_link (*voir figure 8*). En effet, l'odométrie est transformée en base_link, mais il se peut que la base_frame soit initialisée à un autre nom.

Dans le **docker**, faites coulisser et cherchez :

```
\opt\ros\humble\share\slam_toolbox\config\mapper_params_online_async.yaml
```



```
1  slam_toolbox:
2    ros_parameters:
3
4      # Plugin params
5      solver_plugin: solver_plugins::CeresSolver
6      ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
7      ceres_preconditioner: SCHUR_JACOBI
8      ceres_trust_strategy: LEVENBERG_MARQUARDT
9      ceres_dogleg_type: TRADITIONAL_DOGLEG
10     ceres_loss_function: None
11
12     # ROS Parameters
13     odom_frame: odom
14     map_frame: map
15     base_frame: base_link
16     scan_topic: /scan
17     use_map_saver: true
18     mode: mapping #localization
```

FIGURE 1 – Fichier yaml de online_async

2.4 Lancement de la navigation

Assurez-vous que votre fichier : ~/roskit_ws/src/navigation2/nav2_bringup/params/nav2_params.yaml est correct. Vous pouvez en récupérer un valide ici : [Lien GitHub](#).

Lancez la navigation :

```
ros2 launch nav2_bringup navigation_launch.py
```

Dans RViz, utilisez l'outil "Nav2 Goal" pour définir une destination.



FIGURE 2 – Paramètres rviz

2.5 Enregistrement de la carte

Pour visualiser la carte que vous êtes en train de créer allez dans rviz et faite : **ADD -> By Topic -> /map -> Map.**

Cela permet de visualiser le retour du **/map** et de se repérer. Ce qui vérifie que le programme fonctionne bien.

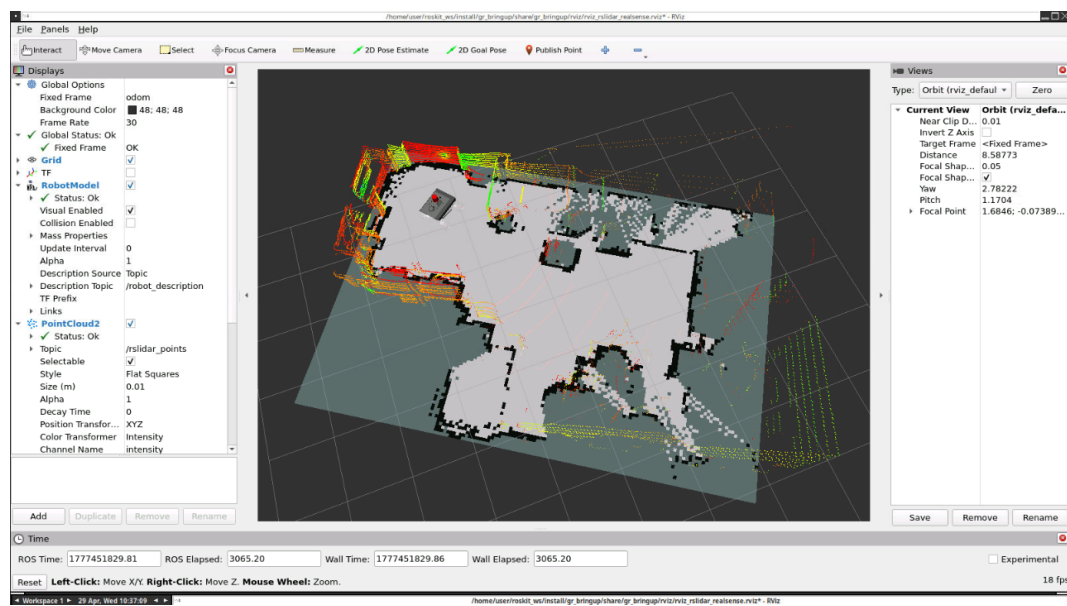


FIGURE 3 – Rendu de la carte au sol

Maintenant, pour voir les trajecotirs du robot et faire de la navigation autonome nous allons utiliser la default view de nav2 :

```
ros2 run rviz2 rviz2 -d ~/roskit_ws/src/navigation2/nav2_bringup/rviz/
nav2_default_view.rviz
```

Vous devriez avoir un rendu avec beaucoup de d'information :

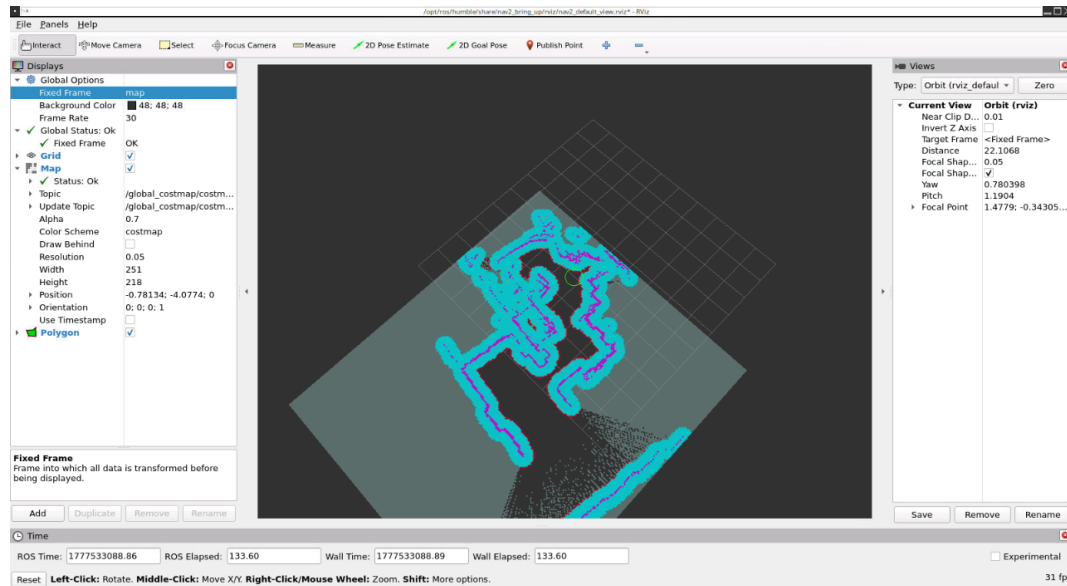


FIGURE 4 – Rendu de la carte au sol

Vous pouvez décocher certaines options pour mieux y voir. Ces options permettent de savoir ce que prend en compte le robot pour effectuer le pathfinding. Si rien ne s'affiche aller dans les topics et ajouter la "map" et le "polygone" du **global costmap**, le "path" et goal si vous le voulez. Faites dérouler la map dans le menu et choisissez "costmap", vous verrez comme sur la figure 11, les bordures. Cela permet de voir clairement si le robot est bloqué ou non. En violet sont les murs, en bleu les murs avec l'inflation, et en rouge le bord de l'inflation.

Si robot submergé : Dans le fichier nav2_params.yaml, allez a global costmap et réduisez **inflation_radius** à 0.55 et le **robot_radius** a 0.35. Maintenant relancer votre nav2_bringup, activez la carte, l'inflation des mur et la taille du robot d'avoir permettre plus de mouvements.

Au bout de quelques rotations, le robot se perd. En effet, l'odométrie est imprécise. Pour corriger ces valeurs voir le le TP sur l'IMU.

Pour sauvegarder votre travail : Vous pouvez effectuer :

```
ros2 run nav2_map_server map_saver_cli -f nom_de_la_carte
```

3 Utilisation de RTABMAP

3.1 Intallation des packages

```
# Dans le docker :
docker exec -it gr-p247-humble-container bash
# Installation des outils
cd ~/roscit_ws/src

# Clonage de Navigation2 (Branche Humble)
git clone https://github.com/introlab/rtabmap.git src/rtabmap
git clone --branch ros2 https://github.com/introlab/rtabmap_ros.git src/rtabmap_ros

rosdep install --from-paths src --ignore-src -r -y

# On desactive les flags qui pourraient forcer le multi-coeur
unset MAKEFLAGS

# Compilation globale limitee a 1 worker, les packets
# etant tres massif, il pourrait faire crash le robot.
colcon build --symlink-install \
  --cmake-args -DCMAKE_BUILD_TYPE=Release \
  --parallel-workers 1

# On lance tous les programmes
source install/setup.bash
(ros2 launch gr_bringup main.launch.py)
```

3.2 Paramétrage RTABMAP

Tout va se passer dans le **launch file**. Rendez-vous dans le dossier dans lequel se trouve le launch file **rtabmap_ros/rtabmap_launch**. Ouvrez le launch file python. Pour commencer on va choisir le mode d’affichage. Cherchez la fonction `generate_launch_description()` aux alentours des lignes 420. Voici les changements à faire et pourquoi les faire, dans l’ordre chronologique. Par soucis de place, les descriptions ci-dessous seront vides, ne pas hésitez à lire les descriptions dans la fonction même.

Visualisation : Pour commencer On choisi le mode et l’affichage. Le mode localisation nécessite une carte, `rtabmap_viz` est le visualiseur par rtab, mais il ne semble pas fonctionner, mais nous utiliserons `rviz`.

Listing 1 – Paramètres d’affichage

```
DeclareLaunchArgument('localization', default_value='false', description='...'),
DeclareLaunchArgument('rtabmap_viz', default_value='false', description='...'),
DeclareLaunchArgument('rviz', default_value='true', description='...'),
```

Temps : Ensuite il va falloir s'assurer de ne pas utiliser le temps de simulation :

Listing 2 – Paramètres de simulation

```
DeclareLaunchArgument('use_sim_time', default_value='false', description='...'),
```

ID des frames : Assurez vous que la frame_id soit sur 'base_link' et que l'odom_frame_id soit sur 'odom'. Pour être de bien récupérer la position du robot.

Listing 3 – ID des frames

```
DeclareLaunchArgument('frame_id', default_value='base_link', description='...'),  
DeclareLaunchArgument('odom_frame_id', default_value='odom', description='...'),
```

File d'attente : Ensuite nous allons augmenter la taille de la file d'attente des topics. Si le message LiDAR attend trop longtemps et que 10 autres messages arrivent entre-temps, il est expulsé de la file avant que son partenaire (la Caméra) n'ait eu le temps d'arriver. Résultat : RTAB-Map ne publie rien.

Listing 4 – Paramètres de file d'attente

```
DeclareLaunchArgument('topic_queue_size', default_value='50',description='...'),  
DeclareLaunchArgument('queue_size', default_value='50', description='...'),
```

Synchronisations des capteurs : Maintenant nous allons activer l'approx_sync pour nous laisser une tolérance quand on décalage possible entre l'envoi des capteurs afin d'assurer une fusion des capteurs. Le paramètre approx_sync_max_interval permet de donner la tolérance maximal.

Listing 5 – Paramètres de synchronisation de capteurs

```
DeclareLaunchArgument('approx_sync', default_value='true', description='...'),  
DeclareLaunchArgument('approx_sync_max_interval', default_value='0.2',  
    description='...'),
```

Topics de caméra : Les prochains paramètres concernent les topics de la caméra. Il va falloir mettre le chemin vers nos topics. Attention à bien prendre le topics avec la caméra de profondeur alignée !

Listing 6 – Paramètres de récupération de caméra

```
DeclareLaunchArgument('rgb_topic', default_value='/camera/camera/color/image_raw',  
    description=''),  
DeclareLaunchArgument('depth_topic', default_value='/camera/camera/  
    aligned_depth_to_color/image_raw', description=''),  
DeclareLaunchArgument('camera_info_topic', default_value='/camera/camera/color/  
    camera_info', description=''),
```


Topic du Lidar : De la même façon, nous allons devoir récupérer lse données Lidar. RTABMAP permet d'utiliser le point cloud 3D fournis par notre lidar. Ici les deux premiers paramètres correspondent à un nuage de points en 2D, similaire à celui utilisé pour slam_toolbox.

Listing 7 – Paramètres de récupération du Lidar

```
DeclareLaunchArgument('subscribe_scan', default_value='false', description=''),
DeclareLaunchArgument('scan_topic', default_value='/scan', description=''),
DeclareLaunchArgument('subscribe_scan_cloud', default_value='true', description=
    ''),
DeclareLaunchArgument('scan_cloud_topic', default_value='/rslidar_points',
    description=''),
DeclareLaunchArgument('scan_normal_k', default_value='0', description=''),
```

Correcteur odométrique : En ce qui concerne l'odométrie, nous n'utiliserons pas l'odométrie visuel mais plutôt l'ICP. En effet, l'ICP est bien plus précise. L'odométrie visuel utilise la caméra (RGB + Depth) et essaie de faire matcher les nuages de points, tandis que l'ICP utilisent le Lidar, qui fonctionne dans le noir et repère plus aisément les murs. De plus, il faut désactiver la publication de l'odom par RTAB, qui crée un flux perturbateur sur lequel le robot se base pour estimer sa position.

Listing 8 – Paramètres de l'odométrie

```
DeclareLaunchArgument('visual_odometry', default_value='false', description='...
    '),
DeclareLaunchArgument('icp_odometry', default_value='true', description='...'),
DeclareLaunchArgument('odom_topic', default_value='odom', description='...'),
DeclareLaunchArgument('vo_frame_id', default_value=LaunchConfiguration('
    odom_topic'), description='...'),
DeclareLaunchArgument('publish_tf_odom', default_value='false', description=''),
```

Topic de l'IMU : Pour finir, en fonction de si nous avons un IMU ou non vous pouvez soit renseigner vos valeurs d'IMU ou désactiver l'option. 'wait_imu_to_init' bloque le démarrage de l'odométrie visuelle/ICP jusqu'à ce que l'IMU ait reçu suffisamment de données pour calculer une orientation initiale fiable.

Listing 9 – Paramètres de récupération de l'IMU

```
DeclareLaunchArgument('imu_topic', default_value='/imu/disabled', description='
    ...'),
DeclareLaunchArgument('wait_imu_to_init', default_value='false', description='')
,
```

Ce qui conclut les paramétrages de RTAB. Vous pourrez faire les tests avec ou sans l'ICP. Avec ou sans IMU.

3.3 Première utilisation RTABMAP

Une fois tous ces paramétrage effectuer nous allons pouvoir utiliser rtabmap. A chaque fois que vous lancerez rtab, votre map sera sauvegardé dans `./ros/rtabmap.db`. Afin de ne pas repartir sur la même carte a chaque fois, si vous voulez effectuer plusieurs essaies. Vous pouvez soit faire **rm ./ros/rtabmap.db** avant de réexécuter votre programme ou alors ajouter un arguments à chaque lancement :

```
ros2 launch rtabmap_launch rtabmap.launch.py rtabmap_args="--delete_db_on_start"
```

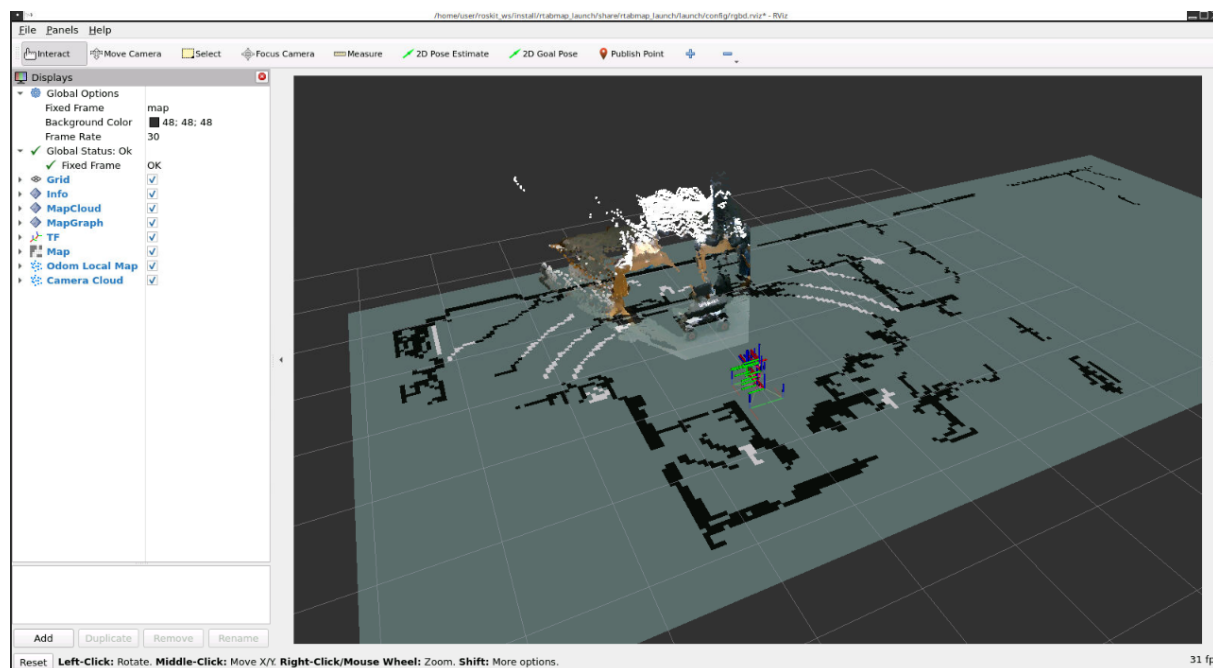


FIGURE 5 – Résultat d'un lancement RTABMAP

Ce que vous devriez voir : La carte au sol avec des murs formés avec le Lidar. Face au robot une reconstruction des images de la caméra avec de la profondeur. Vous pouvez utiliser la télécommande pour déplacer le robot. Je suggère de tourner lentement, en particulier si vous n'avez pas d'IMU ou d'odométrie filtrées. Vous devriez pouvoir reconstruire votre environnement.

4 Navigation

4.1 Identification de problèmes

Comme notre carte est maintenant fournis par rtabmap, nous allons devoir précisé dans le fichier de paramètre. Dans `nav2_params.yaml` ajouter dans le global `costmap`, static_layers, en dessous de `map_subscribe_transient_local` : **`map_topic : /rtabmap/-map`**.

Pour ce qui est de la navigation, on l'utilise de la même façon que précédemment. Avant de faire bouger le robot, visualisez la costmap. Pour se faire, comme d'habitude, ADD cherchez `global_costmap` Map. Vous devriez voir qu'il n'y a pas d'inflation. C'est normal mais c'est dangereux :

```
ros2 launch nav2_bringup navigation_launch.py
```

Pour remédier a ce problème on va légèrement modifier le `nav2_params.yaml`, dans le global costmap et le local costmap **modifier** :

```
obstacle_layer:  
  plugin: "nav2_costmap_2d::ObstacleLayer"  
  enabled: true  
  observation_sources: pointcloud  
  pointcloud:  
    topic: /rslidar_points  
    max_obstacle_height: 2.0  
    min_obstacle_height: 0.1  
    clearing: true  
    marking: true  
    data_type: "PointCloud2"
```

Grace a ces modifications, la navigation est sécurisée. On vient utiliser les données Lidar 3D. On ne passe plus par un scan transformé par **point to laserscan**. De cette façon nav est capable de créer l'inflation. La raison pour laquelle nous n'avons pas fait cela pour le premier TP est que slamtoolbox nécessite spécifiquement un laserscan 2D, il ne comprend pas le **PointCloud2**.

4.2 Fonctionnement :

Avec les modifications effectuées, vous pouvez relancer la navigation, avant de bouger je suggère de vérifier si la costmap est bien paramétrée. Affichez la map du topic de la global costmap, puis changez le color scheme pour costmap.

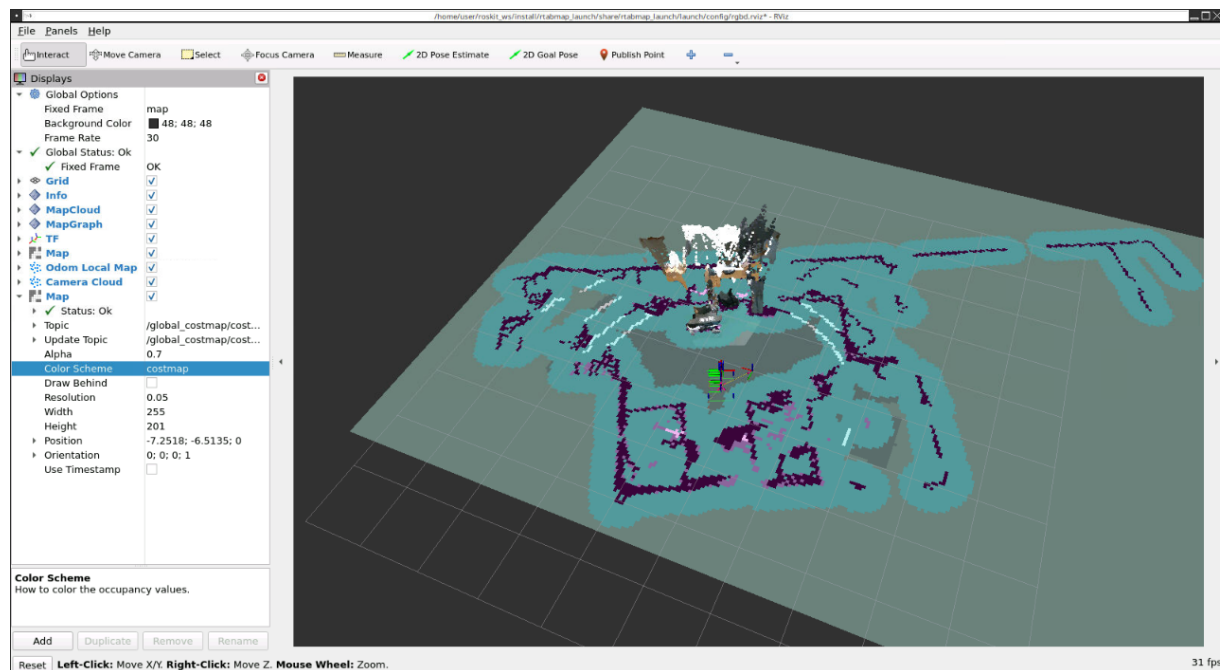


FIGURE 6 – Affichage de la costmap

Vous pouvez modifier légèrement les valeurs afin de rendre un endroit praticable ou de le sécuriser. Encore une fois, les valeurs dans le fichier de paramètres nav2.

4.3 Avec IMU :

Si vous utilisez un EKF, vérifiez que ces topics sont en ordre (CTRL + F) :

Dans rtabmap.launch.py : Cherchez odom_topic, mettez le nom du topic filtrée. Ensuite, imu_topic, de même précisez le nom de votre topic. Si vous n'êtes pas sûr, faites ros2 topic list, cherchez le nom du topic. Avec VSCode vous pouvez copier coller le nom du topic pour être sûr.

Dans nav2_params.yaml : Cherchez encore une fois odom_topic et mettez le nom du topic filtrée.